

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY....3
CLIENT....3
INSTRUCTIONS....3
DEVELOPER....4
1. ALGORITHM CIPHER....4
2. CERTIFICATE GENERATION....4
3. DEPLOY CIPHER....4
4. SECURE COMMUNICATIONS....4
5. SECONDARY TESTING....4
6. FUNCTIONAL TESTING....4
7. SUMMARY....4
8. INDUSTRY STANDARD BEST PRACTICES....4

Document Revision History

Version	Date	Author	Comments
1.0	[Date]	[Your Name]	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Vincent Esposito

1. Algorithm Cipher

My recommendation for a secure and reliable algorithm cipher is AES-256-GCM. AES (Advanced Encryption Standard) is a widely used symmetric encryption algorithm which uses a 256-bit symmetric key, rendering brute force attacks computationally impossible. AES-256-GCM combines a 256-bit symmetric clock cipher with a 128-bit Galois/Counter Mode authentication tag, thanks to AES, to deliver confidentiality and authenticated encryption. Which helps detect unauthorized modifications to encrypted data.

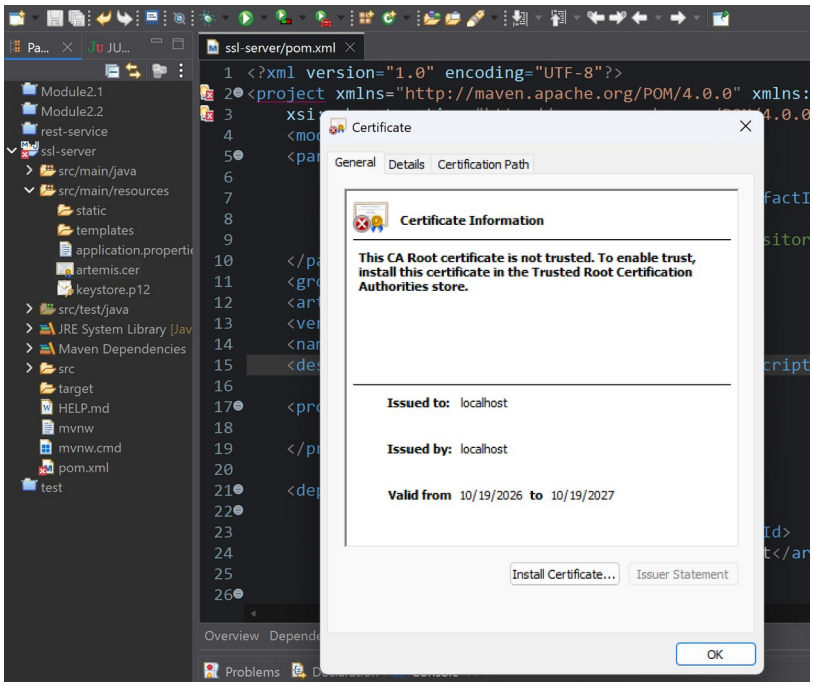
Along with the encryption capabilities of AES-256, I suggest using the SHA-256 hashing algorithm in order to verify data integrity. SHA-256 works by taking input data, like a person's name, and converts it into a 256-bit hash value. This hashed value can then be compared against a trusted checksum to determine if the data was altered in any capacity during transmission. This is excellent for defending against attackers using arbitrary code execution to access private data. Unlike AES encryption, SHA-256 is a one-way function and is not intended to be reversed to retrieve the original data.

AES-256-GCM uses symmetric encryption, which means the same key is used to encrypt and decrypt information. Symmetric encryption tends to perform well when working with large amounts of data, which actually makes it perfect for a financial institution like Artemis. Symmetric encryption differs from asymmetric encryption in that asymmetric encryption uses a public and private key pair to support authentication and secure key establishment. Randomness is another important staple for security as encryption keys need to be generated using cryptographically secure methods to prevent attackers from being able to predict and generate them on their own. AES-GCM also requires a unique nonce for each encryption operation performed with the same key to prevent serious security vulnerabilities.

Encryption algorithms have evolved exponentially since the dawn of computational technology. Older algorithms, such as the Data Encryption Standard (DES) and Triple DES (3DES), have become outdated due to security limitations and inability to compete with modern security threats. AES was adopted around 2001 and has remained the standard in modern security systems.

2. Certificate Generation

Insert a screenshot below of the CER file.



3. Deploy Cipher

Insert a screenshot below of the checksum verification.

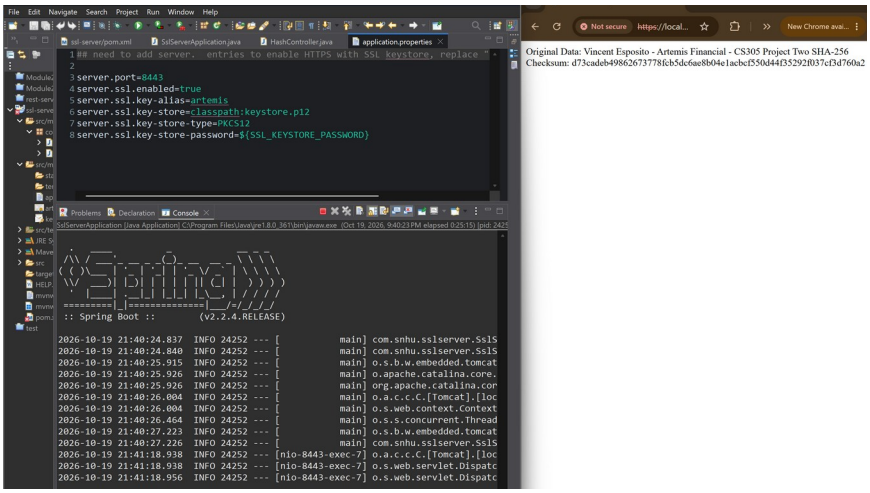


Original Data: Vincent Esposito - Artemis Financial - CS305 Project Two SHA-256 Checksum: d73cadeb49862673778fcb5dc6ae8b04e1acbcf550d44f35292f037cf3d760a2

4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.

The application's configuration was updated to support HTTPS communication using a self-signed certificate and a PKCS12 keystore. The application successfully launched on port 8443, and the /hash endpoint was accessible through HTTPS



5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.

Refactored Code – HashController.java (SslServerApplication.java)

```java
// creating a separate controller like this helps separate
//application startup and checksum
package com.snhu.sslserver;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HashController {

    @GetMapping("/hash")
    public String generateHash() throws NoSuchAlgorithmException {

        // Unique data string containing student information
        String data = "Vincent Esposito - Artemis Financial - CS305 Project Two";

        // Create a SHA-256 hash of our data
        MessageDigest digest = MessageDigest.getInstance("SHA-256");

        byte[] hashBytes = digest.digest(
            data.getBytes(StandardCharsets.UTF_8)
        );

        // Convert the hash into a readable hexadecimal string
        StringBuilder hexString = new StringBuilder();

        for (byte b : hashBytes) {
            hexString.append(String.format("%02x", b & 0xff));
        }
```

```java
        // Create a SHA-256 hash of our data
        MessageDigest digest = MessageDigest.getInstance("SHA-256");

        byte[] hashBytes = digest.digest(
            data.getBytes(StandardCharsets.UTF_8)
        );

        // Convert the hash into a readable hexadecimal string
        StringBuilder hexString = new StringBuilder();

        for (byte b : hashBytes) {
            hexString.append(String.format("%02x", b & 0xff));
        }

        // Return the original data and its checksum
        return "Original Data: " + data
                + "\nSHA-256 Checksum: " + hexString.toString();
    }
}

```

Application.properties

```
## need to add server.  entries to enable HTTPS with SSL keystore, replace "????" with correct entries

server.port=8443
server.ssl.enabled=true
server.ssl.key-alias=artemis
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-type=PKCS12
server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD}
```

Dependency Check Report



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constit
analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, th

How to read the report | Suppressing false positives | Getting Help: github issues

Project: ssl-server

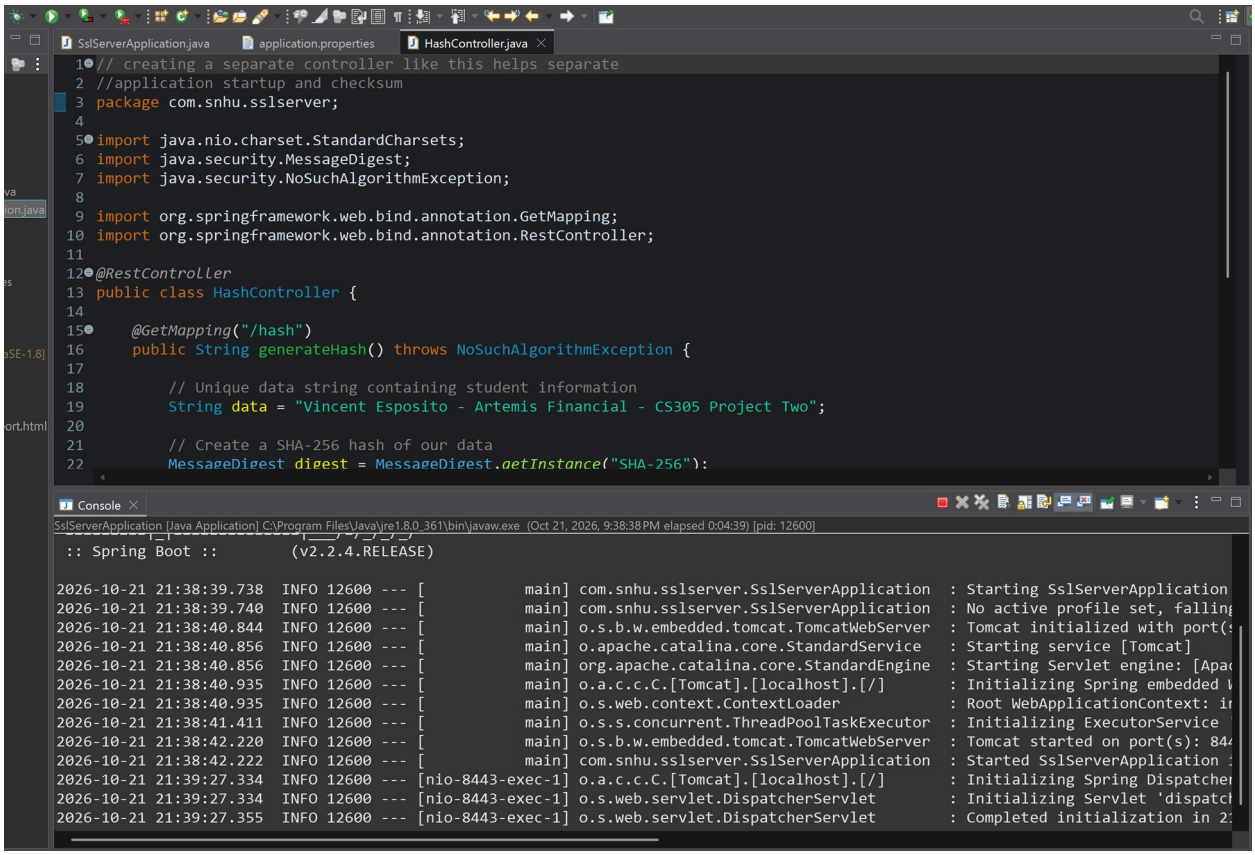
com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.2.2
- *Report Generated On*: Tue, 20 Oct 2026 20:58:08 -0400
- *Dependencies Scanned*: 49 (28 unique)
- *Vulnerable Dependencies*: 14
- *Vulnerabilities Found*: 254
- *Vulnerabilities Suppressed*: 0
- ...

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

Artimis Financial's main objective was to modernize their web application. With modernization, however, comes the need to assess modern security vulnerabilities. I was tasked with improving the applications security through cryptography, client/server communication, APIs, and code quality. I implemented SHA-256 hashing to generate a checksum for verifying data integrity. While also configuring the application to use HTTPS with a self-signed certificate and keystore, providing encrypted communication between the client and server.

I decided the most secure and efficient method for implementing my changes was through a layered security approach which combined cryptographic hashing, secure communication protocols, and security testing. I manually combed through the refactored code before moving onto an additional vulnerability scan using OWASP Dependency-Check. The scan identified 254 vulnerability findings across 14 dependencies, indicating that the application's older libraries still contain security risks that may

require further attention. I feel my implementations are sound, but making the effort to update vulnerable dependencies or refactoring the code in such a way that removed the need for certain dependencies would be a worth wile investment for the company.

8. Industry Standard Best Practices

Industry standard best practices are the backbone of any good software development. It was very important that I applied industry-standard security practices to improve Artemis Financial's application while maintaining its existing functionality. These practices would include SHA-256 hashing through Java's built-in MessageDigest class, configuring HTTPS communication, and protecting the keystore password through an environment variable. I was also constantly performing manual code reviews along with functional testing and vulnerability scanning through OWASP Dependency-Check to identify potential security weaknesses.

Leveraging these practices allowed me to help Artemis achieve their overall goal; updating the application while protecting sensitive customer information, reducing the risk of security breaches, and maintaining customer trust. Secure coding also greatly mitigates the likelihood of costly system outages and subsequent damage to the company's reputation. From here on, I would highly encourage regular security reviews and constantly updating the maintenance guide to ensure the security is able to keep competing with modern threats.

Works Cited:

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)* (FIPS Publication 180-4). https://doi.org/10.6028/NIST.FIPS.180-4

National Institute of Standards and Technology. (2023). *Advanced Encryption Standard (AES)* (FIPS Publication 197). https://doi.org/10.6028/NIST.FIPS.197

Oracle. (n.d.). *Class MessageDigest (Java Platform SE 8).* https://docs.oracle.com/javase/8/docs/api/java/security/MessageDigest.html

OWASP Dependency-Check. (2026, October 20). *Dependency-check report: ssl-server* (Version 12.2.2) [Unpublished HTML security scan report].

OWASP Foundation. (n.d.). *OWASP Dependency-Check.* https://owasp.org/projects/dependency-check/